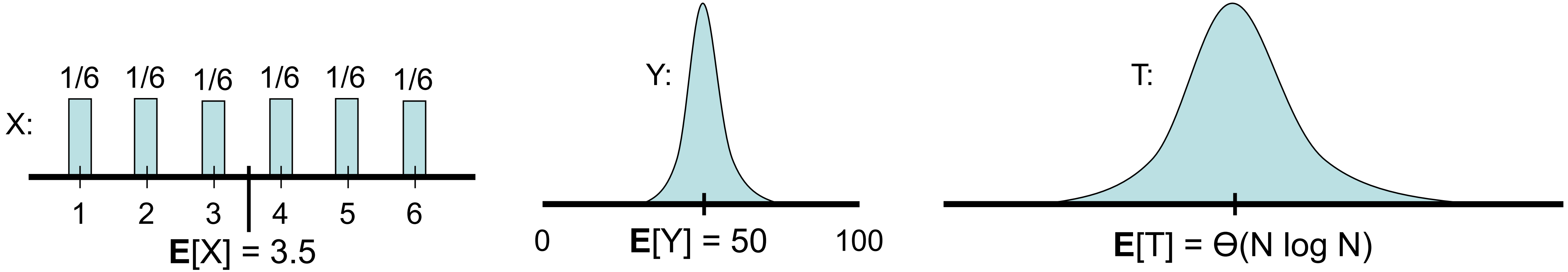
Expected Value and Expected Running Time

Design and Analysis of Algorithms
Brian C. Dean

School of Computing
Clemson University

Random Variables

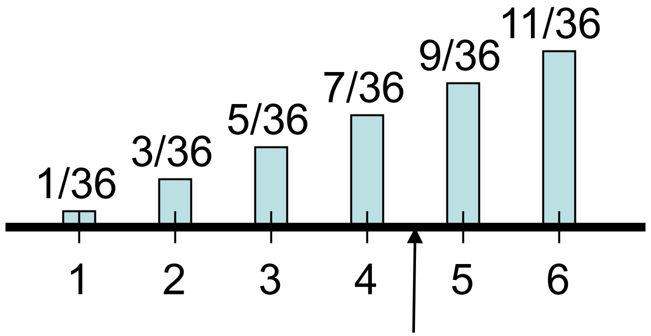
- A variable is “placeholder” for some specific value.
- By contrast, a **random variable** stands for a numeric value that is determined by the outcome of some random experiment.
- Every random variable has an associated probability distribution.
- Examples:
  - Let X be the number we see when we roll a 6-sided die.
  - Let Y be the number of heads in 100 fair coin flips.
  - Let T be the running time of randomized quicksort on a length-N array.



Expected Value

- The expected value **E**[X] of a discrete random variable X is the "center of mass" of X's distribution:

$$\mathbf{E}[X] = \sum_v v \, \mathbf{Pr}[X = v].$$



- **Example:** Let D be the max of two dice rolls.

$$\mathbf{E}[D] = 1(1/36) + 2(3/36) + 3(5/36) + 4(7/36) + 5(9/36) + 6(11/36) = 4\ {}^{17}/_{36}$$

- **Example:** Let H be 1 if we flip heads on a fair coin, 0 otherwise.

$$\mathbf{E}[H] = 1\ \mathbf{Pr}[\text{heads}] + 0\ \mathbf{Pr}[\text{tails}] = \mathbf{Pr}[\text{heads}] = ½$$

Linearity of Expectation

- **E**[] is a **linear** operator:

  E[cX] = c**E**[X] if c is a constant

  E[X + Y] = **E**[X] + **E**[Y]

- The above holds for **any** random variables X and Y, regardless of whether or not they are independent!
- This gives us a very powerful tool for computing expectations of complicated random variables, by decomposing them into sums of simpler random variables (e.g., indicator random variables).
- Example: Let H be the total number of heads in 100 coin flips.

  $H = H_1 + H_2 + \ldots + H_{100}$, where $H_j = 1$ if the j^{th} coin toss comes up heads.

  Now $\mathbf{E}[H] = \mathbf{E}[H_1] + \ldots + \mathbf{E}[H_{100}] = 100(1/2) = 50$.

Linearity of Expectation : Simple Examples

- If N hat-wearing people randomly permute their hats, what is the expected number of people ending up with their original hat?
- If we randomly throw N balls into M bins, what is the expected number of balls landing in a specific bin?
  - Algorithmic application: load balancing by assigning N jobs with processing times $p_1 \ldots p_N$ randomly to M servers.
- Given N points $p_1 \ldots p_N$, suppose you start at a point x and repeatedly move in the direction of a randomly-chosen p_i by replacing x with $x + \varepsilon(p_i - x)$. Were will you ultimately end up?
  - Motivates <u>stochastic</u> gradient descent…

Average Case / Per Element Analysis

- It's easy to build a histogram out of N values $v_1 \ldots v_N$ in O(N) time.
- However, what if the values arrive <u>online</u> and we want to keep the histogram up to date throughout in real time.
- For each new value v_k that arrives…

… normally, just increment appropriate bin count in O(1) time.

… unless v_k is a new maximum, which causes
a full re-build in $\Theta(k)$ time!

- Worst case runtime? $\Theta(N^2)$
- Average case if v_i's arrive in random order? $\Theta(N)$